Ambientes Virtuais Python no VS Code

Guia prático, melhores práticas e troubleshooting — Windows 10 e 11

Público-alvo: iniciantes e intermediários em Python **Pré-requisito:** nenhum conhecimento prévio de linha de comando **Tempo estimado:** 25–40 min (primeira execução) · 3 min (execuções seguintes)

---

1. Por que usar ambiente virtual

Sem isolamento, todas as bibliotecas são instaladas na instalação global do Python. Isso gera três problemas previsíveis:

- **Conflito de versões:** o Projeto A precisa de `pandas 1.5`, o Projeto B de `pandas 2.2`. Sem isolamento, um dos dois quebra.
- **Irreprodutibilidade:** o código roda na sua máquina e falha na do colega, porque ninguém sabe exatamente o que está instalado.
- **Poluição do sistema:** desinstalar dependências vira tentativa e erro.

O ambiente virtual (`venv`) cria uma pasta local com um interpretador Python próprio e um diretório `site-packages` isolado. **Regra fundamental: um ambiente virtual por projeto.**

---

2. Preparação do ambiente (uma vez por máquina)

2.1 Instalar o Python

Recomendação: **instalador oficial do python.org** ou **winget**. Evite a versão da Microsoft Store — ela usa um sandbox de arquivos que causa comportamentos inesperados em projetos reais.

winget install Python.Python.3.12
Se optar pelo instalador gráfico, marque obrigatoriamente:

> **Add python.exe to PATH**
> **py launcher** (instala o `py`, que gerencia múltiplas versões)

Validação:

```
py --version
py -0p        # lista todas as versões instaladas e seus caminhos
```

> **Atenção — o alias fantasma da Microsoft Store:** se `python` abrir a Microsoft Store em vez de rodar o interpretador, desative o atalho em *Configurações → Aplicativos → Configurações avançadas de aplicativo → Aliases de execução de aplicativo* e desligue `python.exe` e `python3.exe`.

2.2 Instalar o VS Code e as extensões

Extensão	Publisher	Função
Python	Microsoft	Detecção de interpretadores, execução, depuração
Pylance	Microsoft	IntelliSense e checagem de tipos (instalada junto com a anterior)
Jupyter	Microsoft	Apenas se o projeto usar notebooks
Ruff	Astral	Linter e formatador (opcional, recomendado)

2.3 Liberar a execução de scripts no PowerShell

Este é **o erro nº 1** de quem usa Windows. O PowerShell bloqueia scripts por padrão, e o comando de ativação do venv é um script `.ps1`.

Abra o PowerShell (não precisa ser como administrador) e execute **uma única vez**:

Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
Confirme com `S` (ou `Y`). Isso permite rodar scripts locais e mantém a exigência de assinatura para scripts baixados da internet — um equilíbrio adequado entre segurança e usabilidade.

2.4 Escolhendo entre múltiplas versões instaladas

Ao criar um ambiente, o VS Code apresenta a lista de interpretadores detectados. Em máquinas com histórico de uso, essa lista costuma ser longa e confusa:

Select a Python installation to create the virtual environment
📁 Enter interpreter path...
Python 3.14.7 ~\AppData\Local\Python\pythoncore-3.14-64\python.exe Global
Python 3.13.15 ~\AppData\Roaming\uv\python\cpython-3.13.15-windows-x86_64-none\python.exe
Python 3.13.15 ~\.local\bin\python3.13.exe
Python 3.12.14 ~\AppData\Roaming\uv\python\cpython-3.12.14-windows-x86_64-none\python.exe

Python 3.12.14 ~\.local\bin\python3.12.exe

Primeiro: cinco entradas não significam cinco Pythons. Neste exemplo há apenas três interpretadores reais.

Padrão do caminho	O que é
`AppData\Local\Python\pythoncore-X.Y-64\`	Instalação oficial via Python Install Manager ou winget
`AppData\Local\Programs\Python\PythonXY\`	Instalação oficial via instalador gráfico (python.org)
`AppData\Roaming\uv\python\cpython-...`	Interpretador baixado e gerenciado pelo `uv`
`~\.local\bin\pythonX.Y.exe`	**Atalho (shim)** para a entrada `uv` correspondente — mesmo executável
`AppData\Local\Microsoft\WindowsApps\`	Alias da Microsoft Store — **evitar** (ver §2.1)
`C:\ProgramData\anaconda3\`	Distribuição Anaconda/Miniconda

Entradas com mesma versão e caminhos `uv` + `.local\bin` são o mesmo interpretador listado duas vezes. Escolher uma ou outra produz ambiente idêntico.

Segundo: o rótulo `Global` não é recomendação. Indica apenas qual interpretador responde quando você digita `python` no terminal sem ambiente ativo.

Critérios de decisão, em ordem de prioridade

1. **O projeto já define a versão?** Verifique `requires-python` no `pyproject.toml`, comentário no `requirements.txt`, `README`, `Dockerfile` ou workflow de CI. Havendo restrição declarada, ela decide — não há o que ponderar.
2. **Onde o código vai rodar?** Se o deploy é um container `python:3.12-slim` ou uma função serverless em 3.12, desenvolva na mesma versão. Divergência entre desenvolvimento e produção é fonte recorrente de bug intermitente e caro de diagnosticar.
3. **As bibliotecas críticas já suportam?** No PyPI, aba *Download files*, procure wheel com a tag correspondente (`cp312`, `cp313`, `cp314`). Sem wheel, o `pip` tenta compilar do código-fonte — no Windows isso exige o Visual C++ Build Tools e falha com frequência.
4. **Sem nenhuma restrição?** Adote **N-1**: uma versão *minor* atrás da mais recente.

Perfil de risco por posição na fila

Posição	Perfil	Indicação
N (mais recente)	Ecossistema majoritário já suporta; dependências de nicho e bindings C ainda apresentam lacunas	Projeto pessoal, experimentação, estudo da linguagem
N-1	Cobertura de wheels praticamente universal; anos de suporte pela frente	**Padrão recomendado para trabalho com prazo**
N-2	Máxima compatibilidade	Stack de dados pesada (PyTorch, GDAL, geoespacial) ou espelhamento de produção legada
N-3 ou anterior	Fim de suporte se aproximando	Apenas manutenção de legado

> **Custo do erro:** trocar a versão de um ambiente virtual leva menos de um minuto (ver §4.5). Não é uma decisão arquitetural — é uma escolha reversível. Não paralise a turma nesse ponto.

---

3. Criando o ambiente virtual

Existem dois caminhos. O primeiro é mais rápido e menos propenso a erro; o segundo ensina o que está acontecendo por baixo.

Caminho A — Pela interface do VS Code

Abra a **pasta do projeto** no VS Code (`Arquivo → Abrir Pasta`). Nunca abra um arquivo `.py` solto.

1. `Ctrl + Shift + P` → digite **Python: Create Environment**
2. Selecione **Venv**
3. Escolha a versão do Python (ex.: `Python 3.12.x`)
4. Se existir `requirements.txt`, o VS Code oferece instalá-lo automaticamente — marque a caixa

O VS Code cria a pasta `.venv`, seleciona o interpretador e adiciona ao `.gitignore` quando aplicável. **Nenhum comando de ativação manual é necessário** nos terminais abertos depois disso.

Caminho B — Pelo terminal (recomendado para entender o mecanismo)

```
# 1. Navegue até a pasta do projeto
cd C:\Projetos\meu-projeto

# 2. Crie o ambiente
py -m venv .venv

# 3. Ative o ambiente
.\.venv\Scripts\Activate.ps1
```

O prompt passa a exibir o prefixo `(.venv)`:

```
(.venv) PS C:\Projetos\meu-projeto>
```

Comandos de ativação por terminal:

Terminal	Comando
PowerShell (padrão do Windows 11)	`.\.venv\Scripts\Activate.ps1`
Prompt de Comando (cmd)	`.venv\Scripts\activate.bat`
Git Bash	`source .venv/Scripts/activate`
WSL / Linux	`source .venv/bin/activate`

Para desativar, em qualquer terminal: `deactivate`

---

4. Ativação e seleção do interpretador

4.1 São duas coisas diferentes

Esta distinção resolve a maior parte da confusão de quem está começando:

	Ativação	**Seleção do interpretador**
Onde atua	No **terminal**	No **editor**
O que faz	Faz `python` e `pip` apontarem para o `.venv`	Faz o Pylance analisar o código contra o `.venv`

	Ativação	Seleção do interpretador
Como se faz	`.\.venv\Scripts\Activate.ps1`	`Ctrl+Shift+P` → *Select Interpreter*
Escopo	Aquela sessão de terminal	Todo o workspace, persistente
Sintoma se faltar	`pip install` vai para o Python global	Imports válidos sublinhados em vermelho

Fazer as duas é o estado correto. Ao selecionar o interpretador no VS Code, **todo terminal aberto a partir daí já nasce ativado** — o `Activate.ps1` manual deixa de ser necessário no dia a dia.

4.2 Selecionando o interpretador no VS Code

1. `Ctrl + Shift + P` → **Python: Select Interpreter**
2. Escolha a entrada marcada com **`('.venv': venv)`** e caminho `.\.venv\Scripts\python.exe`

Validação visual: a barra de status inferior direita deve exibir algo como `3.13.15 ('.venv': venv)`.

4.3 Diagnóstico: o ambiente está realmente ativo?

Rode sempre este comando — é a **fonte de verdade**:

python -c "import sys; print(sys.executable)"

Saída	Interpretação
`D:\meu-projeto\.venv\Scripts\python.exe`	✅ Ativo e correto
`C:\Users\...\AppData\Local\Python\...`	❌ Python global — ambiente não ativo
`C:\Users\...\AppData\Roaming\uv\python\...`	❌ Interpretador base do `uv` — não é o venv
`C:\Users\...\AppData\Local\Microsoft\WindowsApps\`	❌ Alias da Microsoft Store (ver §2.1)

O prefixo `(.venv)` no prompt é indicador secundário, não prova. Prompts customizados — oh-my-posh, starship, temas do Windows Terminal — frequentemente suprimem esse

indicador. Um ambiente pode estar perfeitamente ativo sem exibi-lo. Já o `sys.executable` não mente.

4.4 Fluxo de correção

Se o `sys.executable` apontar para fora do projeto, diagnostique a causa antes de agir:

Test-Path .\.venv\Scripts\python.exe
Retornou `True` — o ambiente existe, só não foi ativado nesta sessão:

.\.venv\Scripts\Activate.ps1

> **Causa mais comum:** o terminal já estava aberto quando o ambiente foi criado. Terminais existentes **não são ativados retroativamente**. Fechar e abrir um novo (`Ctrl + Shift + '`) resolve de forma permanente, desde que o interpretador esteja selecionado conforme §4.2.

Retornou `False` — o ambiente não chegou a ser criado. Volte à §3.

Falhou com erro de execução de scripts — ExecutionPolicy restritiva. Aplique o comando da §2.3.

4.5 Conferindo — e trocando — a versão do ambiente

Criar o venv com `py -m venv` ou `python -m venv` usa o Python **global**, que nem sempre é o desejado. Confira:

python --version
Para criar o ambiente com uma versão específica, invoque o interpretador pelo caminho completo:

& "$env:USERPROFILE\.local\bin\python3.13.exe" -m venv .venv
Se o ambiente já existe na versão errada, recrie — é uma operação de um minuto:

deactivate
Remove-Item -Recurse -Force .venv
& "$env:USERPROFILE\.local\bin\python3.13.exe" -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Não existe "consertar" a versão de um venv existente — o interpretador é copiado/referenciado na criação. Recriar é o procedimento padrão, não uma gambiarra.

---

5. Gerenciando dependências

Instalar pacotes

```
python -m pip install --upgrade pip      # sempre atualize o pip primeiro
python -m pip install pandas requests
```

> Use `python -m pip` em vez de `pip` puro. Isso garante que o pip executado é o do ambiente ativo, eliminando uma classe inteira de erros silenciosos.

Congelar e restaurar

```
# Gerar o manifesto de dependências
python -m pip freeze > requirements.txt

# Restaurar em outra máquina
python -m pip install -r requirements.txt
```

Separando produção de desenvolvimento

Prática recomendada para projetos com mais de um colaborador:

```
requirements.txt        # o que a aplicação precisa para rodar
requirements-dev.txt    # pytest, ruff, black, ipykernel...
python -m pip install -r requirements.txt -r requirements-dev.txt
```

Fixação de versões (pinning)

Notação	**Comportamento**	**Quando usar**
`pandas`	Qualquer versão	Nunca em projeto compartilhado
`pandas>=2.0`	Mínima definida	Bibliotecas que você publica
`pandas==2.2.1`	Exata	**Padrão para aplicações e material didático**
`pandas~=2.2.0`	Permite patches (2.2.x)	Equilíbrio entre estabilidade e correções

Para material educacional, use sempre `==`. A alternativa garante que o tutorial escrito hoje ainda funcione daqui a seis meses.

---

6. Configuração do projeto

`.gitignore`

O ambiente virtual **nunca** vai para o repositório. Ele pesa centenas de MB, contém caminhos absolutos da sua máquina e é integralmente reconstruível a partir do `requirements.txt`.

```
.venv/
__pycache__/
*.py[cod]
.env
.ipynb_checkpoints/
```

`.vscode/settings.json` — persistindo o ambiente entre sessões

Sem esta configuração, todo dia recomeça igual: abrir o projeto, abrir o terminal, descobrir que `python` aponta para outro lugar e rodar o `Activate.ps1` na mão. O arquivo abaixo elimina essa repetição e, por ser versionável, propaga o comportamento para quem clonar o repositório.

O arquivo

Crie `.vscode/settings.json` na raiz do projeto:

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}\\.venv\\Scripts\\python.exe",
  "python.terminal.activateEnvironment": true,
  "python.analysis.typeCheckingMode": "basic",
  "editor.formatOnSave": true,
  "files.exclude": {
    "**/__pycache__": true
  }
}
```

A variável `${workspaceFolder}` resolve para a raiz do projeto em tempo de execução. Por isso o arquivo funciona em qualquer máquina, independentemente de onde o repositório foi clonado — nunca escreva caminho absoluto aqui.

Três formas de criar

Método	**Como**
Terminal	`mkdir .vscode -Force` seguido de `code .vscode\settings.json`

Método	Como
Explorer	Ícone *Novo Arquivo* → digitar `.vscode/settings.json` (a barra cria a pasta junto)
Paleta	`Ctrl+Shift+P` → **Preferences: Open Workspace Settings (JSON)**

> ⚠️ Na paleta, escolha **Workspace**, nunca *User*. A variante *User* altera configurações globais de todos os projetos da máquina.

O arquivo sozinho pode não bastar

Esta é a principal fonte de confusão, e vale entender o mecanismo antes de depurar:

Peça	Papel	Prevalece?
`python.defaultInterpreterPath`	Sugestão usada quando **nenhum** interpretador foi escolhido ainda para o workspace	Não
Python: Select Interpreter	Escolha efetiva, gravada no estado interno do workspace	**Sim**
`python.terminal.activateEnvironment`	Manda ativar o interpretador escolhido em cada terminal novo	—

Se o workspace já tem uma seleção anterior — de um Anaconda, de um Python global, de um venv antigo — o `defaultInterpreterPath` é **ignorado**. Ele é padrão, não ordem.

Por isso o procedimento completo tem dois passos:

1. Criar o `settings.json`
2. `Ctrl + Shift + P` → **Python: Select Interpreter** → entrada `('.venv': venv)`

Não aparecendo a entrada `.venv` na lista, use **Enter interpreter path** e informe o caminho completo (`D:\projeto\.venv\Scripts\python.exe`).

Terminais existentes não são afetados

A configuração se aplica na **criação** do terminal. Sessões já abertas continuam como estavam. Feche todos e abra um novo (`Ctrl + Shift + '`).

Funcionando corretamente, o VS Code injeta sozinho a linha de ativação — você a vê aparecer sem ter digitado:

```
PS D:\projeto> & d:/projeto/.venv/Scripts/Activate.ps1
(.venv) PS D:\projeto>
```

Validando a persistência

O teste que importa não é o de agora, é o de amanhã. Simule:

1. Feche o VS Code por completo
2. Reabra e selecione o projeto em *Arquivo → Abrir Recentes*
3. `Ctrl + Shift + '`
4. Confirme sem digitar nada:

```
python -c "import sys; print(sys.executable)"
```

Passando nesse teste, o `Activate.ps1` some da sua rotina.

O que sobrevive ao fechamento

Item	**Persiste?**
Pasta `.venv` e pacotes instalados	Sim — está em disco
Banco de dados e arquivos do projeto	Sim
Interpretador selecionado	Sim, gravado no workspace
Ativação da sessão de terminal	Não — mas é refeita automaticamente
Servidor de desenvolvimento em execução	Não — precisa subir de novo

Não existe procedimento de "fechar o projeto". Basta fechar a janela: o `deactivate` só faz sentido para sair do ambiente **sem** encerrar o terminal.

Sempre abra a pasta, nunca o arquivo

Configuração de workspace só carrega quando o VS Code abre um diretório. Abrir um `.py` solto (duplo clique no Explorer do Windows, por exemplo) não ativa nada — nem o `settings.json`, nem o interpretador, nem o terminal.

```
cd D:\projeto
code .
```

Versionar, sim

Ao contrário do `.venv`, o `.vscode/settings.json` **deve** ir para o Git. É o que faz alguém que clona o repositório criar seu próprio ambiente e já encontrar o interpretador corretamente

configurado — a diferença entre "funciona na minha máquina" e um projeto que a equipe consegue rodar.

Estrutura de referência

```
meu-projeto/
├── .venv/              ← ignorado pelo Git
├── .vscode/
│   └── settings.json   ← versionado
├── src/
│   └── main.py
├── tests/
├── .gitignore
├── requirements.txt
└── README.md
```

---

7. Melhores práticas — checklist

Nomenclatura e localização

- Use sempre o nome `.venv` (convenção reconhecida automaticamente pelo VS Code e pelo `uv`)
- Mantenha a pasta na **raiz do projeto**, não em um diretório central compartilhado
- Evite caminhos com **espaços, acentos ou cedilha** (`C:\Projetos\` é melhor que `C:\Meus Documentos\Análise`)

Armadilhas específicas do Windows

- **OneDrive / Google Drive:** não crie projetos dentro de pastas sincronizadas. A sincronização de milhares de arquivos do `.venv` gera lentidão, locks de arquivo e corrupção do ambiente. Use `C:\Projetos\` ou `C:\dev\`.
- **Caminho longo (260 caracteres):** projetos aninhados profundamente quebram instalações. Habilite suporte a caminhos longos via `Editor do Registro → HKLM\SYSTEM\CurrentControlSet\Control\FileSystem → LongPathsEnabled = 1`, ou simplesmente mantenha a raiz curta.
- **Antivírus:** Windows Defender pode desacelerar drasticamente a criação de venvs e o `pip install`. Adicione `C:\Projetos\` às exclusões se o desempenho for crítico.

Higiene do ambiente

- Nunca instale pacotes no Python global — se `python -m pip install` rodar sem `(.venv)` no prompt, cancele

- Ao clonar um repositório, o primeiro comando é sempre criar o venv e instalar o `requirements.txt`
- Ambiente corrompido? Delete a pasta `.venv` e recrie. O ambiente é **descartável por natureza** — essa é justamente a vantagem do modelo.

Documentação

- O `README.md` deve conter o bloco de setup pronto para copiar e colar:

Setup

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

---

8. Comparativo de ferramentas de ambiente

Ferramenta	Instalação	Velocidade	Lock file	Indicação
venv	Nativo do Python	Média	Não (manual)	**Padrão. Ensino, projetos simples e médios**
uv	`winget install astral-sh.uv`	Muito alta (10–100×)	Sim	Projetos profissionais; CI/CD
Poetry	Instalador próprio	Média	Sim	Bibliotecas publicadas no PyPI
conda / miniconda	Instalador próprio	Baixa	Sim	Data science com dependências binárias (GDAL, CUDA)
virtualenv	`pip install virtualenv`	Alta	Não	Legado; compatibilidade com Python 2

Ferramenta	Instalação	Velocidade	Lock file	Indicação
pipenv	`pip install pipenv`	Baixa	Sim	Em declínio; sem vantagem clara hoje

Recomendação de adoção:

- **Aprendizagem e material didático:** `venv` — é nativo, não exige instalação adicional e ensina o conceito sem abstrações
- **Projetos de produção:** `uv` — compatível com o ecossistema `pip`/`venv`, com ganho de performance expressivo
- **Data science pesado:** `conda`, pelo gerenciamento de dependências não-Python

---

9. Troubleshooting

Sintoma	Causa provável	Solução
`não pode ser carregado porque a execução de scripts foi desabilitada`	ExecutionPolicy restritiva	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`python` abre a Microsoft Store	Alias de execução do Windows	Desativar em *Aliases de execução de aplicativo*
`ModuleNotFoundError` em pacote recém-instalado	Interpretador errado selecionado	`Python: Select Interpreter` → escolher `.venv`
Terminal não mostra `(.venv)`	Terminal aberto antes da criação, **ou** prompt customizado que suprime o indicador	Validar com `sys.executable` (§4.3) antes de concluir que há problema
`pip install` foi para o Python global	Ambiente não ativado na sessão	Ativar, desinstalar do global e reinstalar no venv
`'py' não é reconhecido`	PATH não configurado	Reinstalar o Python marcando *Add to PATH*

Sintoma	Causa provável	Solução
`settings.json` criado, mas o terminal não ativa	Existe seleção anterior gravada no workspace — `defaultInterpreterPath` é ignorado	Rodar **Python: Select Interpreter** (§6) e reabrir o terminal
`python` aponta para Anaconda após remover outra versão	Anaconda estava em segundo lugar no PATH e subiu à primeira posição	Selecionar o interpretador por workspace (§6) — não é preciso desinstalar nada
Pylance sublinha imports válidos	Cache do servidor de linguagem	`Ctrl+Shift+P` → **Developer: Reload Window**
`pip install` extremamente lento	Antivírus ou pasta sincronizada	Mover projeto para fora do OneDrive; excluir pasta do Defender
Ambiente para de funcionar após mover a pasta	Venv armazena caminhos absolutos	Deletar `.venv` e recriar
Notebook não enxerga o ambiente	Kernel não registrado	`python -m pip install ipykernel` e selecionar o kernel `.venv`

---

10. Verificação de aprendizagem

Considere o conteúdo dominado quando conseguir executar, sem consultar este material:

- [ ] Criar um projeto do zero com `.venv` ativo e interpretador selecionado no VS Code
- [ ] Ler a lista de interpretadores do VS Code e identificar entradas duplicadas
- [ ] Justificar a escolha de uma versão de Python com base em restrição do projeto ou alvo de deploy
- [ ] Configurar um projeto para que o terminal ative o ambiente sozinho após reabrir o VS Code
- [ ] Explicar por que `.venv` não vai para o Git, mas `.vscode/settings.json` vai
- [ ] Reconstruir um ambiente a partir de um `requirements.txt` recebido
- [ ] Identificar, pelo prompt do terminal, se o ambiente está ativo
- [ ] Diagnosticar um `ModuleNotFoundError` distinguindo entre "não instalado" e "interpretador errado"

Referência rápida

```
py -m venv .venv                        # criar
.\.venv\Scripts\Activate.ps1            # ativar (PowerShell)
deactivate                              # desativar
python -m pip install <pacote>          # instalar
python -m pip freeze > requirements.txt   # congelar
python -m pip install -r requirements.txt # restaurar
python -c "import sys; print(sys.executable)"  # verificar
```

Atalhos do VS Code

Ação	**Atalho**
Paleta de comandos	`Ctrl + Shift + P`
Novo terminal	`Ctrl + Shift + '`
Selecionar interpretador	`Ctrl+Shift+P` → *Select Interpreter*
Criar ambiente	`Ctrl+Shift+P` → *Create Environment*